



TECNOLÓGICO
NACIONAL DE MÉXICO®



MAESTRÍA EN CIENCIAS
DE LA COMPUTACIÓN

Tecnológico Nacional de México
Campus Ciudad Guzmán

CLASS PROJECT — UNIT 4

Cart-Pole Project

Subject: Reinforcement Learning

STUDENTS

Ing. Ana Sofia Rentería Palomares
Ing. Scarlett Berenice Alcaraz Muñiz
Ing. Carlos Torres Orozco
Ing. Cristian Misael Torres Martinez

PROFESSOR

Dr. Julio Alberto García Rodríguez



Index

Basic explanation of the Reinforcement Learning workflow used.....	2
Description of the Cart-Pole environment and its objective.....	3
Action Space.....	4
Reward Function.....	4
Termination Conditions.....	5
State Transition Function.....	5
Equations of Motion.....	5
Numerical Integration (Euler Method).....	5
Code organization and explanations.....	6
Screenshots, execution results and plots.....	13

Table index

Table 1: State space.....	4
Table 2: Action space.....	4
Table 3: Parameters of the Cart-Pole environment.....	5

Figure index

Figure 1: Cross-entropy steps.....	3
Figure 2: Cross-entropy steps for Cart-Pole.....	6
Figure 3: Neural network.....	7
Figure 4: Data classes.....	8
Figure 5: Batch generator.....	9
Figure 6: Percentile filter.....	10
Figure 7: Main training loop.....	11
Figure 8: Graphs.....	12
Figure 9: Redrawing of the graphs.....	13
Figure 10: Neural network display.....	14
Figure 11: Training cycle loss and reward values.....	14
Figure 12: Training cycle loss and reward plots.....	15
Figure 13: Cart-Pole animation.....	16
Figure 14: Cart-Pole episodes summary.....	16

Basic explanation of the Reinforcement Learning workflow used.

Introduction:

This document presents a technical guide for the implementation of a reinforcement learning solution for the Cart-Pole environment. Using the cross-entropy method as the core algorithm, it walks through the setup, code structure, and execution of the project. All theoretical content presented in this work is based on *Deep Reinforcement Learning Hands-On* by Maxim Lapan.

Cross-entropy foundations

This method falls into the model-free, policy-based, and on-policy categories. These three characteristics are defined as follows:

- It is a model-free method since it does not build a model of the environment; it simply tells the agent what to do at every step.
- It is a policy-based method as it approximates the policy of the agent, meaning it estimates what actions the agent should carry out at every step.
- It is an on-policy method given that it requires fresh data obtained directly from the environment.

To understand the cross-entropy method, it is necessary to understand the concept of policy. The policy is denoted as $\pi(a|s)$, where a represents actions and s is the current state. A nonlinear function (neural network, NN) produces this policy, which essentially determines, for every observation, which action the agent should take.

The cross-entropy method process can be described as follows: the agent passes an observation from the environment to the NN, obtains a probability distribution over the possible actions, and performs random sampling using the distribution to select an action. This random sampling introduces stochasticity into the agent's behavior. Once an action is selected, it is sent to the environment, which returns the next observation and the reward for the previous action and the loop continues.

The agent operates in episodes, each episode is a sequence of observations received from the environment, actions taken, and rewards obtained. Some episodes will perform better than others. Therefore the cross-entropy method discards the low-performing episodes and trains only on the better ones. This is the main idea of this method.

Cross-entropy steps

The Reinforcement Learning workflow used in this project can be summarized in the following steps based on the information provided by Maxim Lapan:

1. Play N episodes using the current model and environment.
2. Calculate the total reward for every episode and establish a reward boundary.
3. Discard the episodes with a reward below the boundary.

4. Train on the remaining episodes, using observations as the input and the issued actions as the desired output.
5. Repeat.

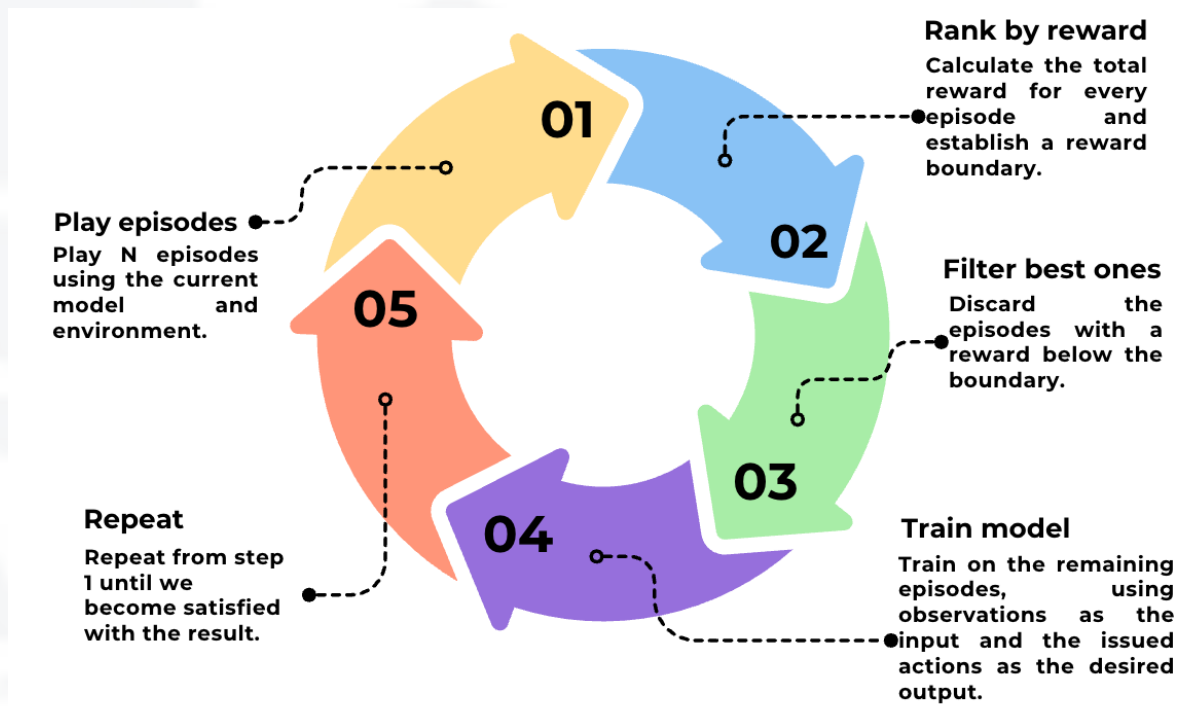


Figure 1: Cross-entropy steps

Description of the Cart-Pole environment and its objective.

CartPole-v1 is a classic control problem included in the Gymnasium library. The environment simulates a physical system in which a pole is attached by an unactuated joint to a cart that moves along a frictionless track. The goal of the agent is to prevent the pole from falling over by applying horizontal forces to the cart.

Objective

The episode continues as long as the pole remains within $\pm 12^\circ$ from vertical and the cart stays within ± 2.4 units from the center of the track. The environment is considered solved when the agent achieves a mean reward greater than 475 over the last 100 episodes. Each time step in which the pole remains balanced contributes a reward of +1.0, so a longer episode directly translates to a higher cumulative reward.

State Space

The state space is continuous and defined as a vector $s \in \mathbb{R}^4$:

$$s = (x \dot{x} \theta \dot{\theta})$$

where each component represents:

- $x \in [-4.8, 4.8]$ — horizontal position of the cart
- $\dot{x} \in (-\infty, +\infty)$ — linear velocity of the cart
- $\theta \in [-0.418, 0.418]$ — angle of the pole from vertical (radians)
- $\dot{\theta} \in (-\infty, +\infty)$ — angular velocity of the pole

Therefore, the complete state space is:

$$S = [-4.8, 4.8] \times \mathbb{R} \times [-0.418, 0.418] \times \mathbb{R} \subset \mathbb{R}^4$$

Index	Variable	Description	Range
0	x	Cart position	$[-4.8, 4.8]$
1	$\dot{x}$	Cart velocity	$(-\infty, +\infty)$
2	θ	Pole angle (rad)	$[-0.418, 0.418]$
3	$\dot{\theta}$	Pole angular velocity	$(-\infty, +\infty)$

Table 1: State space

Action Space

The action space is discrete and defined as:

$$A = \{0, 1\}$$

where:

$$a = \{0 \text{ push the cart to the left } 1 \text{ push the cart to the right}\}$$

Action	Value	Effect
Push left	0	Apply a force of -10 N to the cart
Push right	1	Apply a force of $+10$ N to the cart

Table 2: Action space

Reward Function

The immediate reward at each time step t is:

$$r_t = 1 \quad \forall t \quad \text{while the episode has not ended}$$

and the cumulative reward for the episode (with discount factor $\gamma = 1$) is:

$$R = \sum_{t=0}^T r_t = T$$

where T is the total duration of the episode.

Termination Conditions

The episode terminates if any of the following conditions is met:

$$|x| > 2.4 \quad \text{or} \quad |\theta| > 0.2095 \text{ rad} \quad \text{or} \quad t > 500$$

State Transition Function

Since the CartPole dynamics are based on classical mechanics, the state transition is deterministic:

$$p(s' | s, a) = \delta(s' - f(s, a))$$

where $\delta(\cdot)$ is the Dirac delta and $f(s, a)$ is the system dynamics function. The physical parameters of the environment are:

Parameter	Symbol	Value
Cart mass	M	1.0 kg
Pole mass	m	0.1 kg
Pole length (half)	l	0.5 m
Gravity	g	9.8 m/s ²
Applied force	F	±10 N
Time step	τ	0.02 s

Table 3: Parameters of the Cart-Pole environment

The applied force depends directly on the agent's action:

$$F = \{+10 \text{ N} \quad \text{if } a = 1 \quad -10 \text{ N} \quad \text{if } a = 0$$

Equations of Motion

Applying Newton's laws to the system, the angular and linear accelerations are obtained:

$$\ddot{\theta} = \frac{g \sin\theta + \cos\theta \left(\frac{-F - ml\dot{\theta}^2 \sin\theta}{M + m} \right)}{l \left(\frac{4}{3} - \frac{m \cos^2\theta}{M + m} \right)}$$

$$\ddot{x} = \frac{F + ml(\dot{\theta}^2 \sin\theta - \ddot{\theta} \cos\theta)}{M + m}$$

Numerical Integration (Euler Method)

The next state s_1 is computed by numerically integrating with time step τ :

$$x_{t+1} = x_t + \tau \dot{x}_t$$

$$\dot{x}_{t+1} = \dot{x}_t + \tau \ddot{x}_t$$

$$\theta_{t+1} = \theta_t + \tau \dot{\theta}_t$$

$$\dot{\theta}_{t+1} = \dot{\theta}_t + \tau \ddot{\theta}_t$$

Therefore, the complete transition function is:

$$f(s_t, a_t) = (x_t + \tau \dot{x}_t, \dot{x}_t + \tau \ddot{x}_t, \theta_t + \tau \dot{\theta}_t, \dot{\theta}_t + \tau \ddot{\theta}_t) = s_{t+1}$$

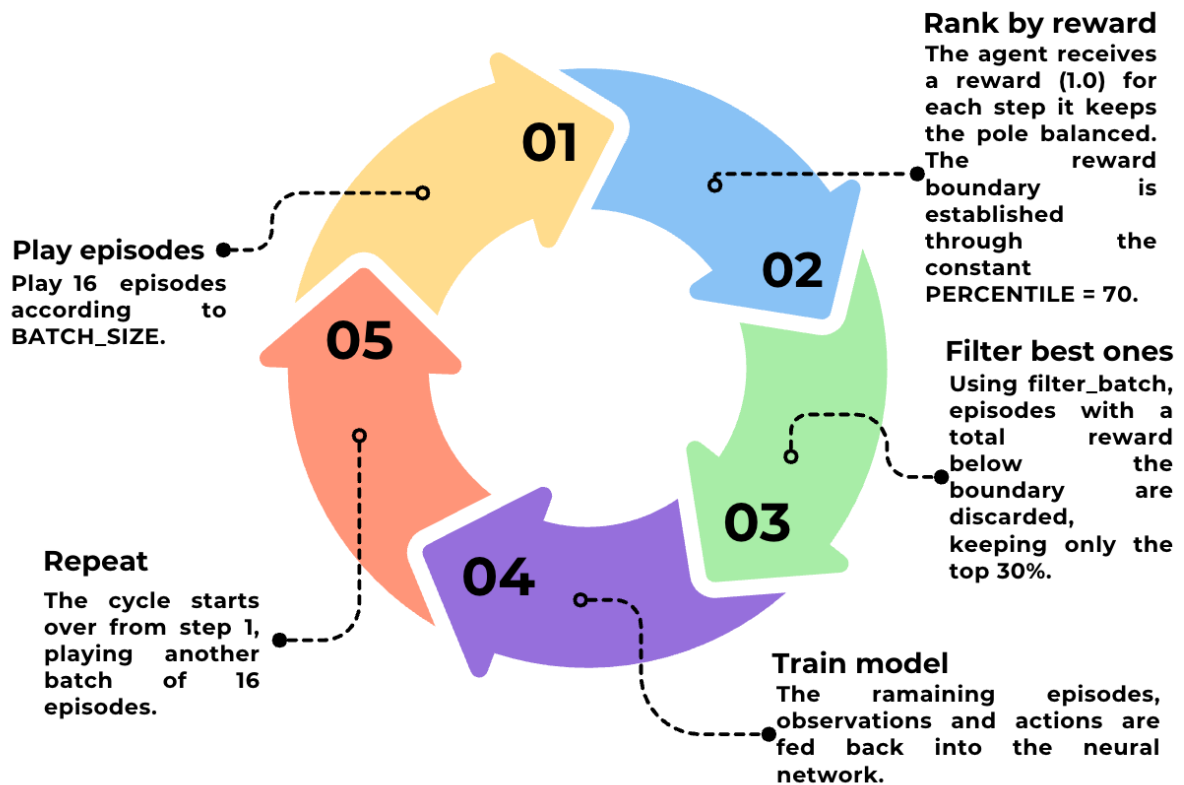


Figure 2: Cross-entropy steps for Cart-Pole

Code organization and explanations

The following section details how the Cart-Pole Python script works. The script was executed under two different Python versions, namely 3.9.5 and 3.10.11, functioning correctly in both cases. The script operates based on the implementation of multiple libraries, which are explained below:

- Numpy: Used for performing calculations in Python. In this project, it will be used to compute percentiles and for random action sampling.
- Gymnasium: A toolkit used for reinforcement learning. This library has been in use since Chapter 2.
- Dataclases: A library used to create data classes in a simple way.
- Typing: A library that helps document what each function receives and returns.
- Torch: The main PyTorch library that performs operations with tensors. These tensors are the primary structure for feeding neural networks.
- Torch.nn: A module that enables the construction of neural networks.
- Torch.optim: A module used for optimization algorithms.



- Matplotlib.pyplot: The main library for creating graphs. In this project, it replaces the TensorBoard from the original code.
- Matplotlib.gridspec: Allows advanced control over the layout and distribution of plots.
- Collections.deque: A double-ended queue with a fixed maximum size from the Python standard library. It is used to store the most recent metric values, automatically discarding the oldest ones for live graphs.

After reviewing the libraries used, we will look at the hyperparameters that were utilized. There are specifically 4 of them, and they are the ones that define the general behavior of the network.

- The first is HIDDEN_SIZE, which indicates the size of the hidden layer of the neural network. In this case, since Cart-Pole is a simple problem with 4 input values, it is given a value of 128.
- The second is BATCH_SIZE, which indicates that 16 complete episodes will be run before each update. We need enough episodes for the percentile to have statistical meaning.
- The next is Percentile, which indicates that from each batch of episodes, only the top 30% will be used for training, while the rest that do not reach a target threshold are discarded.
- Finally, MAX_HISTORY, which solely limits how many iterations are displayed on the real-time graph, in this case, 100.

We will now move on to the neural network itself. It is a simple neural network composed of two linear layers operating with a ReLU activation function. This network does not include a Softmax function at the end, returning unnormalized logits. This is done because, during data collection, the Softmax function is applied manually to convert the logits into probabilities and sample actions. Additionally, during training, CrossEntropyLoss applies softmax internally.

```
# -----  
# NEURAL NETWORK  
# -----  
class Net(nn.Module):  
    def __init__(self, obs_size: int, hidden_size: int, n_actions: int):  
        super(Net, self).__init__()  
        self.net = nn.Sequential(  
            nn.Linear(obs_size, hidden_size),  
            nn.ReLU(),  
            nn.Linear(hidden_size, n_actions)  
        )  
  
    def forward(self, x: torch.Tensor) -> torch.Tensor:  
        return self.net(x)
```

Figure 3: Neural network

We continue with the class definitions. With these, we define that an episode has a total accumulated reward and a list of the steps it produced, which will be used for training if said episode performed well. This could be thought of as saving the agent's complete trajectory rather than saving only its transitions.

```
# -----  
# DATACLASSES  
# -----  
@dataclass  
class EpisodeStep:  
    observation: np.ndarray  
    action: int  
  
@dataclass  
class Episode:  
    reward: float  
    steps: tt.List[EpisodeStep]
```

Figure 4: Data classes

Before going into detail about the functions, it is important to understand the concept of a Batch Generator. In this script, a batch is a set of 16 complete episodes that are collected before performing any network update. Each episode within the batch contains the full sequence of steps the agent took from the moment the environment was reset until the pole fell or the step limit was reached. This approach differs from other RL algorithms such as DQN, where training is performed using individual transitions stored in a replay memory. Here, the agent requires complete episodes because the total reward of each one is what determines whether it was good enough to train on, and that total reward is only known once the episode has ended.

The next part of the code is the `iterate_batches()` function, which is responsible for playing the environment and collecting episodes. It works as a Python generator, meaning it does not return all batches at once, but rather delivers them one by one as each is completed. Inside this function, the Softmax function is applied manually to convert the network's logits into probabilities, and from these probabilities an action is selected stochastically using `np.random.choice()`, which allows the agent to explore different actions instead of always choosing the same one. `unsqueeze(0)` is also used to add the batch dimension to the observation, since the network expects to receive multiple observations at a time rather than just one.

```
# -----  
# BATCH GENERATOR  
# -----  
def iterate_batches(  
    env: gym.Env,  
    net: Net,  
    batch_size: int  
) -> tt.Generator[tt.List[Episode], None, None]:  
    batch = []  
    episode_reward = 0.0  
    episode_steps = []  
    obs, _ = env.reset()  
    sm = nn.Softmax(dim=1)  
  
    while True:  
        obs_v = torch.tensor(obs, dtype=torch.float32)  
        act_probs_v = sm(net(obs_v.unsqueeze(0)))  
        act_probs = act_probs_v.data.numpy()[0]  
        action = np.random.choice(len(act_probs), p=act_probs)  
  
        next_obs, reward, is_done, is_trunc, _ = env.step(action)  
        episode_reward += float(reward)  
        episode_steps.append(EpisodeStep(observation=obs, action=action))  
  
        if is_done or is_trunc:  
            batch.append(Episode(reward=episode_reward, steps=episode_steps))  
            episode_reward = 0.0  
            episode_steps = []  
            next_obs, _ = env.reset()  
  
            if len(batch) == batch_size:  
                yield batch  
                batch = []  
  
        obs = next_obs
```

Figure 5: Batch generator

We continue with the `filter_batch()` function, which is considered the heart of the Cross-Entropy Method. This function receives the collected batch of episodes and applies the percentile filter, computing the minimum reward threshold that an episode must meet to be considered good. Episodes that do not reach this threshold are discarded, and from those that do, all their observation-action pairs are extracted to be used as training data. It is important to note that at this point the notion of how much reward each episode had is lost, since all that matters is whether it was good enough to surpass the threshold.

```
# -----  
# PERCENTILE FILTER  
# -----  
def filter_batch(  
    batch: tt.List[Episode],  
    percentile: float  
) -> tt.Tuple[torch.FloatTensor, torch.LongTensor, float, float]:  
    rewards = [e.reward for e in batch]  
    reward_bound = float(np.percentile(rewards, percentile))  
    reward_mean = float(np.mean(rewards))  
  
    train_obs: tt.List[np.ndarray] = []  
    train_act: tt.List[int] = []  
  
    for episode in batch:  
        if episode.reward < reward_bound:  
            continue  
        train_obs.extend(step.observation for step in episode.steps)  
        train_act.extend(step.action for step in episode.steps)  
  
    train_obs_v = torch.FloatTensor(np.vstack(train_obs))  
    train_act_v = torch.LongTensor(train_act)  
    return train_obs_v, train_act_v, reward_bound, reward_mean
```

Figure 6: Percentile filter

Next, we have the main training loop, which is where all the pieces of the program come together. On each iteration, `iterate_batches()` plays the environment and produces a batch of 16 episodes; then `filter_batch()` discards the poor-performing ones and returns the observation-action pairs from the good ones. With this data, the neural network learns to replicate exactly those actions in those states using `CrossEntropyLoss` as the loss function. This could be considered supervised learning, since no future rewards or state values are computed, the only thing done in this phase is teaching the network to repeat the best behavior. In this project, the loop ends when the reward exceeds 475, given that there is a maximum of 500 steps.

```
for iter_no, batch in enumerate(iterate_batches(env, net, BATCH_SIZE)):
    obs_v, acts_v, reward_b, reward_m = filter_batch(batch, PERCENTILE)

    optimizer.zero_grad()
    action_scores_v = net(obs_v)
    loss_v = objective(action_scores_v, acts_v)
    loss_v.backward()
    optimizer.step()

    loss_val = loss_v.item()

    print(f"{iter_no:3d}: loss={loss_val:.3f} "
          f"reward_mean={reward_m:.1f} "
          f"reward_bound={reward_b:.1f}")

    # Update history
    hist_iters.append(iter_no)
    hist_means.append(reward_m)
    hist_losses.append(loss_val)
    hist_bounds.append(reward_b)

    # Redraw plot every iteration
    update_plot(
        fig, ax_reward, ax_loss, ax_bound,
        list(hist_iters),
        list(hist_means),
        list(hist_losses),
        list(hist_bounds),
    )

    if reward_m > 475:
        print("\n Solved!")
        break
```

Figure 7: Main training loop

For the management and creation of the graphs, two functions are used: `setup_plot()` and `update_plot()`. The first is responsible for creating the matplotlib window with three subplots arranged in a two-row, two-column layout. The top graph spans both columns and displays the mean reward per iteration, including a yellow dashed line marking the target of 475. In the bottom row, the left column shows the training loss and the right column shows the 70th percentile threshold.

```
def setup_plot():
    """Creates the matplotlib window with 3 subplots."""
    plt.style.use("dark_background")
    fig = plt.figure(figsize=(12, 7))
    fig.suptitle("Cross-Entropy Method – CartPole-v1", fontsize=14, color="white")

    gs = gridspec.GridSpec(2, 2, figure=fig, hspace=0.45, wspace=0.35)

    ax_reward = fig.add_subplot(gs[0, :]) # Row 0, both columns
    ax_loss = fig.add_subplot(gs[1, 0]) # Row 1, left column
    ax_bound = fig.add_subplot(gs[1, 1]) # Row 1, right column

    # Mean reward
    ax_reward.set_title("Mean Reward per Iteration", color="white")
    ax_reward.set_xlabel("Iteration")
    ax_reward.set_ylabel("Mean Reward")
    ax_reward.axhline(y=475, color="yellow", linestyle="--", linewidth=1, label="Goal (475)")
    ax_reward.legend(loc="upper left")

    # Loss
    ax_loss.set_title("Loss (Cross-Entropy Loss)", color="white")
    ax_loss.set_xlabel("Iteration")
    ax_loss.set_ylabel("Loss")

    # Reward bound
    ax_bound.set_title("Percentile 70 Threshold", color="white")
    ax_bound.set_xlabel("Iteration")
    ax_bound.set_ylabel("Reward Bound")

    plt.ion() # Interactive mode: allows updates without blocking
    plt.show()
    return fig, ax_reward, ax_loss, ax_bound
```

Figure 8: Graphs

The second function, `update_plot()`, is responsible for redrawing all three graphs on each training iteration with the most recent data. Each graph is cleared and redrawn with the updated values, also adding a semi-transparent color fill beneath each line to improve readability. The colors used are light blue for the mean reward, red for the loss, and green for the percentile threshold.

```
def update_plot(fig, ax_reward, ax_loss, ax_bound,
                iters, means, losses, bounds):
    """Redraws the plots with the latest data."""
    ax_reward.clear()
    ax_reward.set_title("Mean Reward per Iteration", color="white")
    ax_reward.set_xlabel("Iteration")
    ax_reward.set_ylabel("Mean Reward")
    ax_reward.axhline(y=475, color="yellow", linestyle="--", linewidth=1, label="Goal (475)")
    ax_reward.plot(iters, means, color="#00d4ff", linewidth=2, label="Mean Reward")
    ax_reward.fill_between(iters, means, alpha=0.15, color="#00d4ff")
    ax_reward.legend(loc="upper left")

    ax_loss.clear()
    ax_loss.set_title("Loss (Cross-Entropy Loss)", color="white")
    ax_loss.set_xlabel("Iteration")
    ax_loss.set_ylabel("Loss")
    ax_loss.plot(iters, losses, color="#ff6b6b", linewidth=2)
    ax_loss.fill_between(iters, losses, alpha=0.15, color="#ff6b6b")

    ax_bound.clear()
    ax_bound.set_title("Percentile 70 Threshold", color="white")
    ax_bound.set_xlabel("Iteration")
    ax_bound.set_ylabel("Reward Bound")
    ax_bound.plot(iters, bounds, color="#a8ff78", linewidth=2)
    ax_bound.fill_between(iters, bounds, alpha=0.15, color="#a8ff78")

    fig.canvas.draw()
    fig.canvas.flush_events()
```

Figure 9: Redrawing of the graphs

Finally, the `watch_agent()` function is responsible for opening the animation of the already-trained agent, inspired by Chapter 2 of the book where the `render_mode="human"` parameter is introduced. The agent plays 3 complete episodes and prints the number of steps and the reward obtained in each one, allowing the user to visually observe how well it learned to balance the pole.

Screenshots, execution results and plots.

Execution results and plots:

During the execution of the code, the first thing shown after executing the code is the Neural Network (Net or NN), it displays the sequential architecture module designed for data transformation. The initial Linear layer maps the predetermined input features into a hidden dimension of 128 units to extract initial feature combinations. The subsequent ReLU activation function that introduces non-linearity by filtering out negative values, enabling the network to model complex relationships. Finally, the second Linear layer maps these processed hidden features back down to match the designated output dimension of 2, producing the final values used for decision-making.

```
Net(  
  (net): Sequential(  
    (0): Linear(in_features=4, out_features=128, bias=True)  
    (1): ReLU()  
    (2): Linear(in_features=128, out_features=2, bias=True)  
  )  
)
```

Figure 10: Neural network display

The next thing that print is basically the training cycle of the NN, first we print the iteration number, then the losses, the mean reward of the batch and finally the reward boundary.

```
32: loss=0.567  reward_mean=151.4  reward_bound=163.0  
33: loss=0.564  reward_mean=168.9  reward_bound=183.5  
34: loss=0.547  reward_mean=174.8  reward_bound=175.5  
35: loss=0.544  reward_mean=243.2  reward_bound=282.0  
36: loss=0.544  reward_mean=302.4  reward_bound=356.5  
37: loss=0.546  reward_mean=258.1  reward_bound=317.5  
38: loss=0.547  reward_mean=293.8  reward_bound=352.0  
39: loss=0.536  reward_mean=323.1  reward_bound=393.5  
40: loss=0.538  reward_mean=380.9  reward_bound=494.0  
41: loss=0.535  reward_mean=391.1  reward_bound=500.0  
42: loss=0.519  reward_mean=417.2  reward_bound=500.0  
43: loss=0.522  reward_mean=393.9  reward_bound=460.5  
44: loss=0.524  reward_mean=399.7  reward_bound=500.0  
45: loss=0.527  reward_mean=407.1  reward_bound=500.0  
46: loss=0.519  reward_mean=448.1  reward_bound=500.0  
47: loss=0.511  reward_mean=386.7  reward_bound=500.0  
48: loss=0.520  reward_mean=486.9  reward_bound=500.0
```

Figure 11: Training cycle loss and reward values

The same values are used to draw the plot of the values during the training cycle.

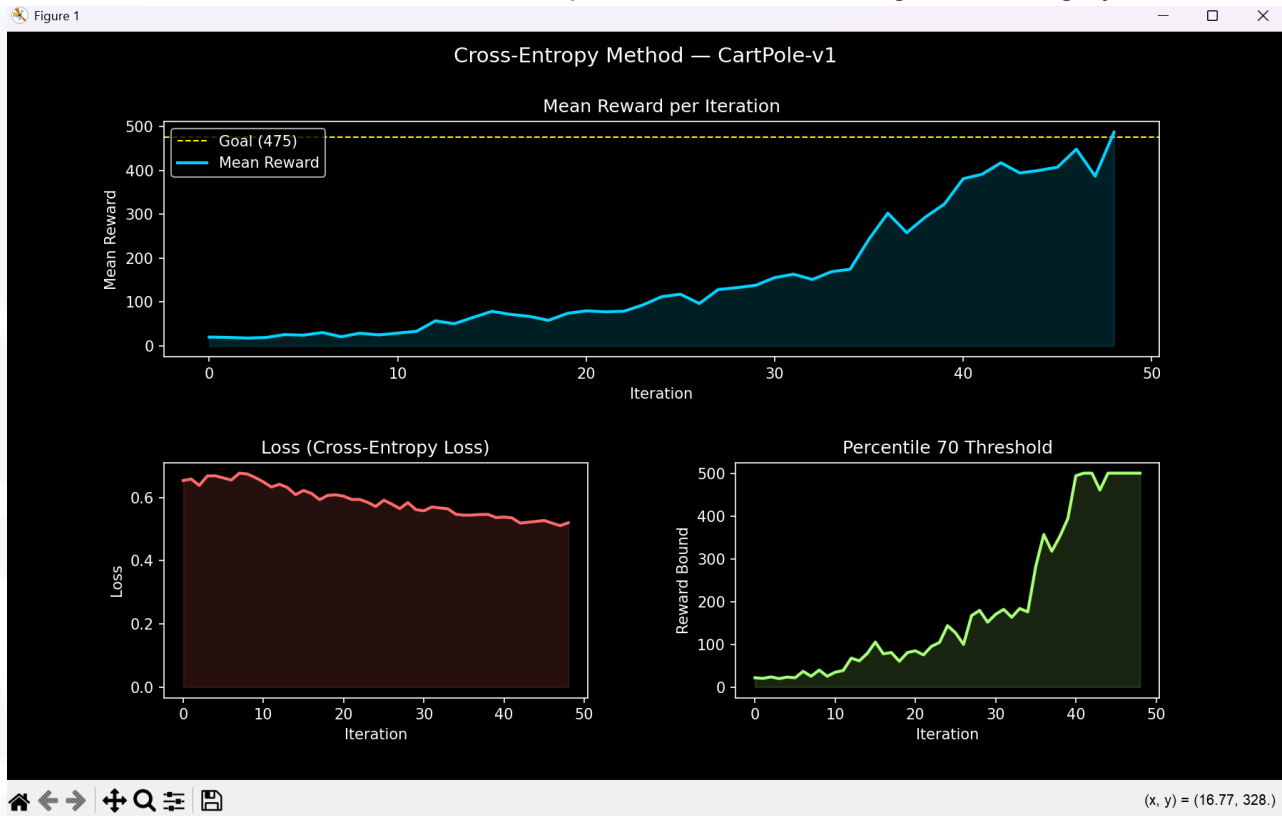


Figure 12: Training cycle loss and reward plots

The previous figure presents three plots. The top plot illustrates the mean reward obtained per iteration, surpassing the threshold of 475 toward the end of training, which indicates that the model was sufficiently trained to approach the maximum reward value of 500. Additionally, the two lower plots represent the Cross-Entropy Loss across iterations (left) and the reward values achieved throughout the training process (right).

After finishing the training cycle, the program will play 3 episodes, at this point, after the model has trained enough times, the cart will balance the pole for an infinitely long period of time, so the length of the episode has been limited to 500 steps each.

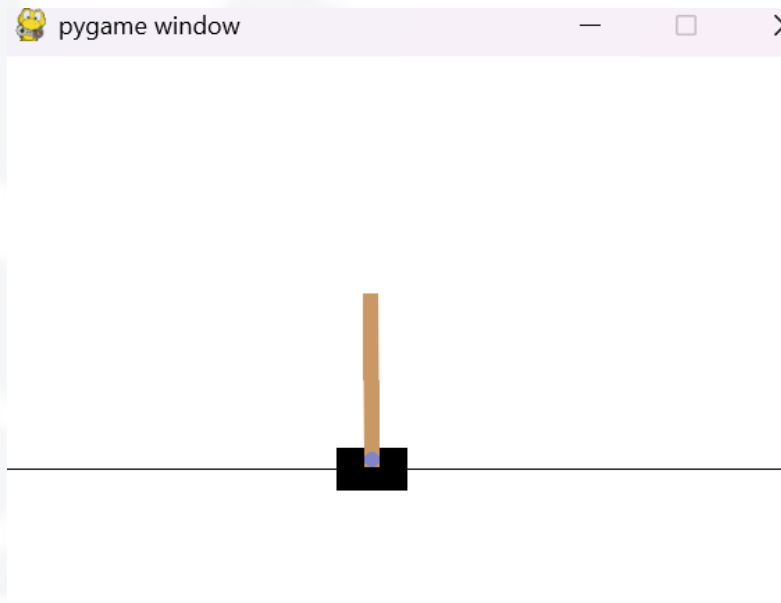


Figure 13: Cart-Pole animation

Every episode is played and if it reaches 500 steps it cuts to the next one, until all episodes are played out.

✓ Solved!

🎮 Opening trained agent animation...
Playing 3 episodes. Close the window to exit.

Episode 1:	500 steps		Reward:	500
Episode 2:	500 steps		Reward:	500
Episode 3:	500 steps		Reward:	500

Figure 14: Cart-Pole episodes summary

Conclusion:

The Cart-Pole program represents a clear example of how reinforcement learning can be applied to scenarios related to real-world problems. In this approach, the interaction between the agent and its environment is first simulated through a training cycle, allowing the model to be trained until the desired performance is achieved before being animated, which would represent a real life case scenario. Solving this type of problem

through reinforcement learning can help reduce costs and improve efficiency in specific tasks.

In the case of Cart-Pole, one potential real-world application is enabling artificial intelligence systems to learn how to stabilize complex structures through simulated trial-and-error processes, without requiring explicit knowledge of the underlying physical laws. Although the current model is still limited in its ability to handle complex three-dimensional environments, it represents a solid first approximation to applied reinforcement learning.

As future work, the training policies, limits, and scope of the Cart-Pole simulation could be further refined. Additionally, a third dimension could be incorporated into the environment in order to more closely resemble real-world problems and applications.